AIIT
AI Interview Training

코드 열람 가이드

파일·함수·메소드 사전과 찾아 읽는 법

AI 1차 채용면접 자동화 플랫폼 · Team AIIT · 2026-11-10 프로그램 기준

이 문서는 `huge/` 프로그램 안에 **어떤 파일이 있고, 그 안의 함수가 무슨 일을 하고, 누가 그 함수를 부르는지**를 쉽게 풀어 쓴 안내서입니다. 코드를 처음 보는 사람도 따라올 수 있게 **비유 → 실제 이름** 순서로 설명합니다. (2026-11-10 코드 기준 — 질문 글 검사·질문 적용 범위·공고 기반 질문 후보·AI 공고 초안 그래프 반영)

> **이렇게 읽으세요** 1장에서 '함수'가 무엇인지 알고 → 2장에서 가게 지도를 보고 → 3장에서 이야기 하나를 따라가고 → 4·5장에서 궁금한 파일을 찾아보세요. 함수 전체 목록은 따로 된 **프로그램 명세서**(`프로그램명세서`)에 있습니다. 사전처럼 찾아볼 때만 쓰면 됩니다.

---

1. 코드 읽기 첫걸음

1-1. 함수는 '자판기'예요

자판기에 **동전과 버튼(넣는 것)** 을 주면 **음료(나오는 것)** 가 나오죠. 함수도 똑같습니다.



그림. 함수는 자판기

```
def hashPassword(password):        # 이름: hashPassword, 넣는 것: password
    return hasher.hash(password)   # 나오는 것: 알아볼 수 없게 갈아 놓은 글
```

- `def` 는 "지금부터 자판기를 만든다"는 뜻이에요.
- 괄호 안 `password` 를 **인자**라고 불러요. 자판기에 넣는 재료예요.
- `return` 다음이 **반환값**, 자판기에서 나오는 결과예요.
- 다른 곳에서 `hashPassword("abc123")` 처럼 쓰면 자판기를 **부른다(호출한다)** 고 해요.
- `hashPassword` 는 🏠 우리가 만든 자판기지만, 그 안의 `hasher.hash` 는 📦 남이 만든 공구(argon2 라이브러리)예요. 누구 것인지 가리는 법은 1-6절에 있어요.

> **원래 뜻:** 함수(function)는 이름이 붙은 코드 묶음으로, 인자를 받아 정해진 일을 하고 반환값을 돌려준다.

> 🔧 **전문 설명** — 함수 정의는 `def 이름(매개변수) -> 반환형식:` 꼴이며, 이 프로젝트는 타입 힌트를 붙여 `def hashPassword(password: str) -> str` 처럼 씁니다. 호출 시 넘기는 실제 값이 인자(argument), 정의의 이름이 매개변수(parameter)입니다. Python 함수는 일급 객체라 📦

`Depends(requireAdmin)`, `graph.add_node("ask", ask)` 처럼 **이름만 넘겨 다른 코드가 나중에 부르게** 할 수 있습니다 — 그래서 일부 함수는 코드 안에 직접 부르는 곳이 없습니다.

1-2. 클래스는 '붕어빵 틀', 객체는 '붕어빵'



그림. 클래스는 틀, 객체는 그 틀로 만든 하나하나

- **클래스**는 틀이에요. `LiveSession` 은 "면접 한 건을 진행하는 직원" 틀입니다.
- 지원자 민지가 면접을 시작하면 이 틀로 **민지 전용 직원(객체)** 이 한 명 만들어져요. 지수가 시작하면 지수 전용 직원이 또 생기고요.
- 그 직원이 할 줄 아는 일이 **메소드**예요. `send`(화면에 말 전하기), `watchdog`(연결 지켜보기) 같은 것들이죠.
- 메소드 안의 `self` 는 "나 자신"이에요. `self.send(...)` = "내가 화면에 전한다".

> **원래 뜻:** 클래스는 데이터(속성)와 그 데이터를 다루는 함수(메소드)를 묶은 설계도이고, 객체는 그 설계도로 만든 실제 하나다.

> 🔧 **전문 설명** — `class LiveSession:` 의 `__init__` 이 생성자이고, `LiveSession(ws, applicationId, sessionId)` 호출이 인스턴스를 만듭니다. 인스턴스 속성(`self.state` , `self.deadline`)은 객체마다 따로 있습니다. 이 프로젝트는 상속을 거의 쓰지 않고, 쓰는 곳은 틀이 요구할 때뿐입니다: `Base(DeclarativeBase)`(SQLAlchemy 모델), `BaseModel`(Pydantic 스키마), `TypeDecorator`(암호화 칸), `Protocol`(`graph.Runner` — 구현을 강제하지 않는 구조적 인터페이스).

1-3. 그 밖에 자주 보이는 것

보이는 모양	쉬운 뜻	원래 뜻
`async def` / `await`	"라면 물 끓는 동안 다른 일 하기". AI 대답처럼 오래 기다릴 때 쉬어 가며 다른 일을 해요	비동기 함수. `await` 지점에서 실행을 양보한다
`@router.post("/auth/login")`	자판기에 붙인 **주소 이름표**. 이 주소로 요청이 오면 이 함수가 일해요	장식자(decorator). 함수를 등록하거나 감싼다
함수 안의 함수	큰 기계 안의 작은 부품	중첩 함수(nested function)
`raise ApiError(...)`	"이건 안 돼요!" 하고 멈추며 이유를 알려 줘요	예외를 발생시켜 실행을 중단한다
대문자로 시작하는 화면 함수 `Modal`	레고 블록 같은 **화면 조각**	React 컴포넌트

보이는 모양	쉬운 뜻	원래 뜻
`use` 로 시작하는 `useApp`	블록에 꽂는 **센서**. 로그인 정보 같은 걸 꺼내 줘요	React 훅(hook)

> 🔧 **전문 설명** — FastAPI 는 `async def` 핸들러를 이벤트 루프에서, 일반 `def` 핸들러를 스레드 풀에서 실행합니다. WebSocket 면접(`LiveSession`)은 한 연결 안에서 수신·heartbeat 감시·자막 중계를 🐍 `asyncio` 작업 여러 개로 동시에 돌립니다. React 쪽은 Next.js App Router 의 클라이언트 컴포넌트 (`"use client"`)이고, 상태는 📦 `useState` ·📦 `useEffect` , 전역 상태는 Context(`Providers`)로 나눕니다.

1-4. 이름만 봐도 하는 일이 보여요

이 프로젝트는 이름 짓는 규칙이 있어서, 이름 끝만 봐도 대충 알 수 있어요.

이름 모양	하는 일	예
`require…`	조건을 검사해서 아니면 막아요 (문지기)	`requireAdmin` — 관리자가 아니면 막음
`…Or404`	찾아보고 없으면 "없어요(404)"	`getPostingOr404`
`…View`	장부(DB) 내용을 화면에 보여 줄 모양으로 바꿔요	`postingView`
`…Task`	뒷주방(작업자)이 하는 오래 걸리는 일	`evaluateTask` — 채점
`is…` / `has…`	예/아니오로 대답해요	`isEmptyAnswer` — 답변이 비었나?
`UPPER_SNAKE`	바뀌지 않는 숫자·글 (상수)	`DEVICE_END_GRACE_SEC = 3`

> 🔧 **전문 설명** — 명명 규칙은 CLAUDE.md 5.1: Python 함수·변수 camelCase(PEP 8 의 snake_case 대신 프로젝트 규칙), 클래스 PascalCase, 상수 UPPER_SNAKE_CASE. DB 칸·API JSON 키는 TRD/API 명세서를 따라 snake_case 이며, SQLAlchemy 의 `process_bind_param` 처럼 라이브러리가 정한 이름은 예외입니다.

1-5. 함수를 찾는 법 (VS Code)

하고 싶은 일	누를 키
파일 열기	`Ctrl+P` 누르고 파일 이름
지금 파일의 함수 목록 보기	`Ctrl+Shift+O`
프로젝트 전체에서 함수 이름으로 찾기	`Ctrl+T`
이 함수가 **어디 있는지** 가기	이름 위에서 `F12`
이 함수를 **누가 부르는지** 보기	이름 위에서 `Shift+F12`
원래 있던 곳으로 돌아오기	`Alt+←`

팁: 화면 주소와 파일은 짝이 있어요. 주소 `/mypage` → 파일 `frontend/app/mypage/page.tsx` . 주소 `/apply/123/consent` → `frontend/app/apply/[applicationId]/consent/page.tsx` (`[ ]` 는 '바뀌는 자리').

> 🔧 **전문 설명** — VS Code 의 `F12` / `Shift+F12` 는 Pylance(Python)·TypeScript 언어 서버의 정적 분석을 씁니다. 문자열로 등록된 Celery 작업(`"huge.sweep"`)이나 그래프 노드처럼 이름으로 연결된 곳은 참조

찾기에 안 잡히므로 `Ctrl+Shift+F` 문자열 검색을 함께 쓰세요.

1-6. 이 함수는 누가 만들었을까?

코드에 나오는 함수가 전부 우리가 만든 건 아니에요. 요리할 때 **내 손**, **집에 원래 있던 공구**, **가게에서 사 온 공구**, **내가 직접 만든 공구**를 섞어 쓰는 것과 같아요.

표시	누구 것?	비유	이 프로젝트의 예	설명은 어디서 찾나
	**내장** — 언어에 처음부터 붙어 있음	내 손	`len` , `print` , `open` , `KeyError` · (JS) `JSON.parse` , `Math.round`	Python / JavaScript 공식 문서
	**표준 라이브러리** — Python 을 깔면 같이 오는 공구함	집 공구함	`secrets` , `hashlib` , `datetime` , `asyncio` , `decimal`	Python 공식 문서 '표준 라이브러리'
	**브라우저 기능** — 브라우저가 화면 코드에 빌려줌	건물에 달린 전기·수도	`fetch` , `getUserMedia` , `MediaRecorder` , `sendBeacon`	MDN 웹 문서
	**외부 라이브러리** — 따로 설치한 남의 공구	가게에서 사 온 공구	FastAPI, SQLAlchemy, LangGraph, argon2 · React, Next.js	그 라이브러리의 문서
	**우리가 만든 함수** — `huge/` 안에 `def` · `function` 이 있음	직접 만든 공구	`hashPassword` , `ownedApplication` , `api`	이 가이드, `F12`

구분하는 법 — 파일 맨 위 `import` 줄을 보세요.

```
import secrets                          # 🐍 설치 안 했는데 있음 → 표준
from argon2 import PasswordHasher       # 📦 requirements.txt 에 있음 → 외부
from app.models import User             # 🏠 app 으로 시작 → 우리 것
from ..timeutil import now              # 🏠 점(.)으로 시작 → 우리 것 (옆 파일)

len(password)                           # 🧰 import 없이 바로 씀 → 내장
```

- `import` 없이 바로 쓰는데 이 파일에 `def` 도 없다? → 🧰 **내장**이에요.
- 이름이 `app.` 이나 `.` 으로 시작한다? → 🏠 **우리 것**이에요.
- 나머지는 `backend/requirements.txt`(화면은 `frontend/package.json`)에 있으면 📦 **외부**, 없으면 🐍 **표준**이에요.
- 화면 코드(`.tsx`)는 `@/` 나 `./` 로 시작하면 🏠, `react` · `next/...` 처럼 이름으로 시작하면 📦, `import` 없이 쓰는 `fetch` · `navigator` 는 🌐 예요.

이 가이드에서는: 프로그램 명세서 표에 적힌 함수는 **모두** 🏠 **우리가 만든 것**이에요. 바깥 것은 명세서의 **"가져다 쓰는 모듈"** 줄과, 4·5장 🔧 전문 설명에서 이름 앞에 붙은 표시(🧰 🐍 🌐 📦)로 알 수 있어요. 표시가 없는 이름은 우리 것이에요.

원래 뜻: 내장 함수(built-in)는 import 없이 쓰는 함수, 표준 라이브러리(standard library)는 언어와 함께 배포되는 모듈 모음, 외부 라이브러리(third-party)는 따로 설치하는 패키지다.

> 🩹 **전문 설명** — Python 내장은 `builtins` 모듈(`dir(__builtins__)`)이고, 표준 라이브러리 목록은 `sys.stdlib_module_names` (3.10+)로 확인할 수 있습니다. 국내 교재(『점프 투 파이썬』 등)는 표준 라이브러리를 **'외장 함수'** 라고 부르기도 해서, pip 패키지를 뜻하는 '외부 라이브러리'와 헷갈리기 쉽습니다 — 이 가이드는 '외장'이라는 말 대신 **표준 / 외부**로 나눕니다. import 이름과 설치 이름이 다른 경우가 있습니다: `fitz` = pymupdf, `argon2` = argon2-cffi, `langchain_openai` = langchain-openai. 외부 라이브러리 함수는 버전에 따라 동작이 바뀔 수 있어 `requirements.txt` 에 버전을 고정(`fastapi==0.118.0`)합니다. FastAPI 의 `Depends` , SQLAlchemy 의 `TypeDecorator` 처럼 **외부 틀이 우리 함수를 대신 불러 주는** 경우가 많아, 프로그램 명세서의 '부르는 곳 `—`' 은 대개 이 📦 틀이 부르는 함수입니다.

---

2. 가게 지도 — 파일들이 사는 곳

이 프로그램을 **AI 면접관이 있는 가게**라고 생각해 보세요. 손님(지원자)이 오면 여러 사람이 차례로 일을 넘겨줍니다.



그림. 요청 하나가 지나가는 층

가게 자리	하는 일	폴더
🪑 **홀** (손님이 보는 곳)	화면, 버튼, 입력 칸	`frontend/app` , `frontend/components`
🛎️ **카운터** (주문 받기)	주소별 요청 받기, 로그인·권한 확인	`backend/app/routers`
🍳 **주방** (요리하기)	질문 만들기, 이력서 읽기, 채점 같은 실제 일	`backend/app/services`
🎙️ **면접실**	면접을 실시간으로 진행	`backend/app/interview`
📏 **규칙판**	상태 이동·점수 계산 규칙 (DB도 AI도 안 씀)	`backend/app/domain`
📒 **장부**	어떤 표에 무엇을 적는지	`backend/app/models.py`
🌙 **뒷주방**	오래 걸리는 일·시간표 일	`backend/app/worker.py` , `jobs.py`

제일 중요한 약속: 홀은 카운터에게만 말하고, 카운터는 주방에 일을 넘기고, 주방은 규칙판과 장부를 봐요. 서로 자기 일만 하니까 고장 났을 때 어디를 고칠지 바로 알 수 있어요.

> 🔧 **전문 설명** — 계층형 구조(Layered Architecture): **표현(Next.js) → API(FastAPI 라우터) → 응용 서비스 → 도메인 → 영속성(SQLAlchemy/PostgreSQL)**. 의존은 한 방향(위 → 아래)이고, `domain/` 은 DB·네트워크를 import 하지 않는 순수 함수라 단위 테스트가 빠릅니다. 오래 걸리는 일은 메시지 큐(Redis 브로커)를 거쳐 Celery 작업자 프로세스(worker · worker-eval · worker-sandbox)가 처리해 API 응답 시간을 지킵니다.

---

3. 이야기로 따라가기 — 민지의 하루

지원자 **민지**가 지원하고 면접을 보고 결과를 받기까지, 어떤 함수들이 차례로 일하는지 따라가 봐요. `→` 는 "다음 사람에게 넘긴다(부른다)"는 뜻이에요.

📝 장면 1. 민지가 이력서를 제출해요

1. 민지가 확인 화면에서 **[제출하기]** 를 눌러요. → 화면 `Confirm` 이 `api()` 로 카운터에 주문을 보내요.
2. 카운터 `submitResume` 가 받아요. "작성 중" 상태에서 "준비됨"으로 바꿔도 되는지 규칙판 `transition` 에게 물어봐요.
3. 괜찮으면 `scheduleCompose` 에게 "민지 질문 10개 준비해 줘" 하고 주방에 맡겨요. 민지는 기다리지 않고 다음 화면으로 가요.
4. 주방의 `compose` 가 질문 10칸을 하나씩 채워요.
   - `similar` 가 비슷한 기존 질문 3개를 찾아 **예시**로 펼쳐 놓고 (RAG),
   - AI(`structured`)가 민지 이력서에 맞는 새 질문을 만들고,
   - `checkGenerated` 가 "질문 모양이 맞나? 이력서에 근거가 있나? 예시를 베끼지 않았나?" 검사해요.
   - 글 검사관 `textProblem` 도 봐요: 이상한 글자가 붙었나? "간단히 답해 주세요" 같은 **답하는 방법 지시**가 있나? **한 번에 두 가지**를 묻나? 말끝이 "~느꼈는가요"처럼 어색하나? 걸리면 "이게 문제였어" 하고 알려 주며 다시 만들게 해요.
   - 3번 실패하면 `bankFallback` 이 **질문은행**에서 골라 와요. 그것도 없으면 예비 질문, 마지막엔 고정 문장이에요.
   - 질문은행에서 고를 때 `candidates` 는 **이 공고에 쓸 수 있는 질문**만 봐요. 다른 공고 전용 회사 질문이나 다른 직무 질문은 빼요(적용 범위).
5. 다 끝나면 장부에 "질문 준비 완료(ready)"라고 적어요.

> 🔧 **전문 설명** — `POST /applications/{id}/resume` → `submitResume` 가 `transition(drafting → ready)` 후 커밋하고 `scheduleCompose` 로 `asyncio` 작업을 띄웁니다(응답은 즉시). `compose` 는 LangGraph `StateGraph` 로 10칸(`SLOT_ORDER`)을 채우며, RAG 단계에서 BGE-M3 1024차원 임베딩으로 pgvector 코사인 검색을 하고, LLM 출력은 JSON 스키마(`RagQuestion`)로 강제·Pydantic 재검증합니다. 글 검사는 `domain/question_text.py` 의 정규식 규칙(찌꺼기 `junk_text` , 답변 방식 지시 `meta_phrase` , 두 가지 묻기 `compound_question` , 어색한 말끝 `awkward_ending` 등)이고, 실패 코드를 `PROBLEM_TEXT` 문장으로 바꿔 이전 질문과 함께 다음 프롬프트에 넣습니다(자기 수정 루프). 질문은행 후보는

`posting_id` · `job_code` 가 비었거나 이 공고·이 직무와 같은 행만 고릅니다(C-17). 검증(`checkGenerated`) 실패가 `question_regen_limit` 번이거나 시간 예산(60초)을 넘기면 질문은행 → 예비 질문 → 고정 문장 순으로 내려가는 **단계적 대체(graceful degradation)** 구조입니다.

🚶 장면 2. 민지가 면접을 봐요

1. 민지가 **[면접 시작]** → 카운터 `startInterview` 가 받아요.
   - `requirePrecheck` : 환경 점검을 15분 안에 통과했나?
   - `tryAcquire` : 지금 면접 자리가 남았나? (놀이기구 번호표)
2. 화면 `Exam` 이 **WebSocket**(끊지 않는 전화선)을 열어요. → `interviewSocket` → 민지 전용 직원 `LiveSession` 이 생겨요.
3. 직원은 순서도(`buildInterviewGraph`)대로 움직여요.
   - `sendQuestion` : 질문 음성과 자막을 보내요.
   - 민지가 말하면 `onAudio` 가 소리를 모으고,
   - `transcribeWithRetry` 가 소리를 글로 바꾸고(STT), `saveAnswer` 가 장부에 적어요.
   - `buildFollowupGraph` : 답이 애매하면 꼬리질문을 하나 더 만들어요.
   - "~한 적이 있나요?"처럼 **예/아니오로 답하는 질문**에 민지가 "네" 하고 짧게만 답하면, `yesNoFollowup` 이 AI 에게 묻지도 않고 바로 "그렇게 답하신 이유를 구체적으로 말씀해 주세요."라고 물어요.
4. 그동안 `watchdog` 은 1초마다 "전화선이 살아 있나?" 지켜봐요.
5. 10문제가 끝나면 `finishInterview` → (코딩테스트가 있으면) `enterCoding` , (없으면) `endWith(completed)` → `finish` .

🔧 **전문 설명** — `startInterview` 는 한 트랜잭션 안에서 점검 유효성(`precheck_valid_until`) 확인, Redis Lua 원자 연산으로 동시 면접 자리 확보(`tryAcquire`), 상태 전이를 처리합니다. 이후 WebSocket(`/ws/interview/{sessionId}`)에서 `LiveSession` 이 LangGraph 체크포인트(PostgreSQL saver, `thread_id = 지원 ID`)와 함께 그래프를 돌려, 재연결 시 같은 지점부터 이어집니다. 답변 음성은 16kHz mono PCM16 바이너리 프레임으로 받고, STT 실패는 `stt_retry_limit` 만큼 재시도합니다. 꼬리질문 판단 전 `yesNoFollowup` 이 `isYesNo(본질문)` 이고 답이 `YES_NO_SHORT_ANSWER` (30자)보다 짧으면 LLM 호출 없이 고정 꼬리질문(reason_code `vague`)을 정하고, 길면 `followupPrompt` 가 '이유가 없으면 이유를 물어라'를 덧붙입니다. 꼬리질문 글도 `validateFollowup` 에서 본질문과 같은 `textProblem` 검사를 거칩니다.

⚡ 장면 3. 갑자기 인터넷이 끊기면?

1. `watchdog` 이 15초 동안 "살아 있어" 신호를 못 받아요.
2. 규칙판 `judgeDisconnect` 에게 물어요: "민지가 나가기 직전에 **'나가요' 쪽지**를 남겼나?"
   - 쪽지 없음 → 인터넷 문제(장애) → `pauseForFault` → `markInterrupted` . 답변은 저장되고, 나중에 이어서 볼 수 있어요.
   - 쪽지 있음 → 민지가 창을 닫은 것 → `endWith(ended)` . 응시가 끝나요.

> 🔧 **전문 설명** — 끊김 판정은 `heartbeat_interval_sec × heartbeat_miss_limit` (5초 × 3 = 15초) 무응답, 또는 장치 종료 + `DEVICE_END_GRACE_SEC` (3초) 대기 뒤 수행합니다. 브라우저는 `pagehide` / `beforeunload` 에서 🌐 `navigator.sendBeacon` 으로 unload 이벤트를 남기고, `judgeDisconnect` 가 `unload_lookback_sec` 안의 이벤트 유무로 `fault` / `not_fault` 를 반환합니다. 판정 근거가 없으면 fault — **불확실성은 지원자에게 유리하게** 라는 설계 원칙입니다.

🧮 장면 4. 채점하고 결과를 알려 줘요

1. `finish` → `queueEvaluation` → `enqueueEvaluation`: 뒷주방에 "채점해 줘" 쪽지를 넣어요.
2. 뒷주방의 `evaluateTask` → `evaluate` 가 시작해요.
   - `redact`: AI가 보기 전에 이름·연락처를 지워요.
   - `scoreQuestion`: 질문마다 AI(4B)가 항목별 0~5점과 **근거 문장**을 내요.
   - `matchQuote`: 그 근거가 **민지 답변에 정말 있는지** 코드가 확인해요 (90% 이상 같으면 인정).
   - `computeScores`: 점수 계산은 **AI가 아니라 코드**가 해요. 총점 ≥ 합격 기준이면 1차 합격.
   - `writeReason`: AI(8B)가 "왜 이 결과인지" 이유 글을 써요. 결과 자체는 못 바꿔요.
   - 이상한 점이 있으면 `NeedsReview` → 관리자가 직접 보는 **평가 지연**으로 가요.
3. 5분마다 `sweepTask` → `releaseDue`: 공개 시각(마감 다음 날 10:00)이 지난 결과를 공개하고 알림을 만들어요. → `queueMail` 로 메일도 보내요.

> 🔧 **전문 설명** — 채점은 커밋 뒤 큐잉(`enqueueEvaluation`)하고, 브로커 장애 시 5분 주기 `sweepTask` 가 누락분(`findUnscored`)을 재투입하는 **at-least-once + 멱등 잠금(`eval:lock:{id}`)** 구조입니다. T5(문항 채점, Qwen3 4B, temperature 0) 결과의 근거 인용은 `matchQuote` 로 원문 대조(정규화 후 `QUOTE_MATCH_RATIO = 0.9`)하고, 점수는 🐍 `Decimal` · 🐍 `ROUND_HALF_UP` 으로 코드가 계산합니다. T10(결과 이유, Qwen3 8B)은 이미 정해진 판정을 입력으로 받아 설명만 생성하며, 검증 실패 2회면 `NeedsReview` (평가 지연)로 사람에게 넘깁니다.

---

4. 백엔드 파일 소개 — "이 파일은 누구?"

각 파일을 **가게의 한 사람**으로 소개합니다. 중요한 함수만 골랐고, 전체 목록은 프로그램 명세서에 있어요.

🚪 입구와 공통 도구

`main.py` — **가게 문 여는 사람** 가게(서버)를 켤 때 필요한 걸 다 연결해요.

- `lifespan` — 켤 때: 장부 표 만들기, 직무 목록 넣기, 1분 정리 일 시작. 끌 때: 정리.
- `recoverAfterRestart` — 껐다 켰을 때, 하다 만 질문 준비를 다시 시작하고 자리 수를 맞춰요.
- `warmUp` — 첫 손님이 기다리지 않게 AI를 미리 깨워요 (첫 호출은 45초 걸렸어요).
- `UploadSizeGuard` — 너무 큰 이력서 파일은 문 앞에서 돌려보내요.

🔧 **전문 설명** — FastAPI 앱 팩토리. `lifespan` (async context manager)에서 📦 `create_all` + `migrate.addColumns`, 체크포인트 saver 열기, `jobs.runForever` 백그라운드 작업 등록. `UploadSizeGuard` 는 ASGI 미들웨어로 `Content-Length` 를 보고 본문을 읽기 전에 411/413 으로 거절합니다(메모리 보호). 예외 처리기(`apiErrorHandler` 등)와 8개 라우터를 📦 `include_router` 로 붙입니다.

`config.py` — 설정 메모장

- `loadEnv` — 컴퓨터 환경 변수(DB 주소, 암호 열쇠 등)를 읽어요.
- `SETTING_DEFAULTS` — 이어가기 3번, heartbeat 5초 같은 기본값 모음이에요.

🔧 **전문 설명** — `Env` dataclass 에 환경 변수를 한 번 읽어 둡니다(12-factor 설정). `SETTING_DEFAULTS` 는 `{키: (운영값, 개발값)}` 이고 실제 값은 DB `app_settings` 가 우선합니다 (`settings_store.getSetting`).

`deps.py` — 문지기들

- `currentUser` — 로그인 안 했으면 "로그인하세요(401)".
- `requireAdmin` / `requireApplicant` — 관리자만, 지원자만 들어오게 해요.
- `ownedApplication` — **내 지원서만** 볼 수 있어요. 남의 것이면 "없어요(404)"라고 해서 있는지조차 숨겨요.

🔧 **전문 설명** — FastAPI 의존성 주입(📦 `Depends`) 함수 사슬: `optionalUser → currentUser → activeUser → requireAdmin/requireApplicant` . `ownedApplication` 은 소유자 불일치 시 403 이 아닌 404 를 돌려 **자원 존재 여부 노출(IDOR 탐색)** 을 막고, PostgreSQL 에서는 `SELECT … FOR UPDATE` 행 잠금을 겁니다.

`security.py` — 금고지기

- `hashPassword` / `verifyPassword` — 비밀번호를 되돌릴 수 없게 갈아서(Argon2) 저장하고, 로그인할 때 맞는지만 확인해요.
- `createSession` / `resolveSession` — 로그인하면 **출입 팔찌(쿠키)** 를 주고, 다음 요청 때 팔찌로 누군지 알아봐요.

🔧 **전문 설명** — 비밀번호는 `argon2-cffi` 의 📦 `PasswordHasher` (Argon2id, 메모리 하드 KDF). 세션 토큰은 🐍 `secrets.token_urlsafe(32)` 로 만들고 DB 에는 **SHA-256 해시만** 저장해 DB 유출 시 세션 탈취를 막습니다. 쿠키는 HttpOnly(스크립트 접근 차단), 만료는 `session_idle_hours` 슬라이딩. `safeNext` 는 로그인 뒤 이동 경로를 허용 목록으로 제한해 오픈 리다이렉트를 막습니다.

`crypto.py` — 자물쇠 장인

- `encryptText` / `decryptText` — 답변·이력서 같은 민감한 글을 열쇠로 잠그고 풀어요 (AES-256-GCM).
- `EncryptedText` — 장부 칸에 붙이면 **저장할 때 자동으로 잠그고, 읽을 때 자동으로 풀어** 줘요.

> 🔗 **전문 설명** — AES-256-GCM(AEAD): 96비트 무작위 nonce + 암호문 + 인증 태그를 base64 로 저장해 **기밀성과 무결성(변조 탐지)** 을 함께 보장합니다. `EncryptedText` / `EncryptedJson` 은 SQLAlchemy 🟫 `TypeDecorator` 로 ORM 경계에서 투명하게 암복호화합니다. 키(`APP_ENCRYPTION_KEY` , 32바이트)가 없으면 기동을 거부(fail-closed)합니다. `subjectHash` 는 동의자 식별에 HMAC 을 써 이메일 사전 대입으로 되돌리지 못하게 합니다.

`errors.py` — 안내 방송

- `ApiError` — "왜 안 되는지"를 `{code, message}` 한 가지 모양으로 화면에 알려 줘요.

> 🔗 **전문 설명** — 도메인 예외 `ApiError(status, code, message)` 를 전역 예외 처리기가 `{"code", "message"}` JSON 으로 바꿉니다(TRD 4-3). Pydantic 검증 오류는 422 `VALIDATION_ERROR` 한 줄로, 예상 못 한 예외는 내부 정보를 숨긴 500 으로 응답합니다.

`timeutil.py` — 시계

- `now` — 지금 한국 시각. `releaseAtFor` — 결과 공개 시각(마감 다음 날 10:00).

> 🔗 **전문 설명** — 모든 시각은 `Asia/Seoul` tz-aware 🐍 `datetime` . SQLite 는 시간대를 저장하지 않아 `asKst` 가 naive 값을 KST 로 해석해 테스트(SQLite)와 운영(PostgreSQL `timestamptz`)의 비교 결과를 맞춥니다.

`settings_store.py` — 운영 설정 서랍

- `getSetting` / `setSetting` — 관리자가 바꾼 설정을 읽고 써요. 없으면 기본값.

> 🔗 **전문 설명** — 키-값 설정 테이블(`app_settings` , 값은 `{"v": …}` JSON). `setSetting` 은 변경 감사 로그를 함께 남깁니다. 알 수 없는 키는 🧰 `KeyError` 로 거부합니다.

`models.py` — 장부 양식 DB에 어떤 표가 있는지 정한 파일이에요. `User` (회원), `JobPosting` (공고), `Application` (지원서), `Answer` (답변), `Result` (결과) 등 28개 표가 있어요. 프로그램 명세서 3-3절에서 표 이름과 칸을 볼 수 있어요.

> 🔗 **전문 설명** — SQLAlchemy 2.0 선언형 매핑(🟫 `Mapped[...]` , 🟫 `mapped_column`). 기본 키는 UUID 문자열(`newId`), 민감 칸은 `EncryptedText` . 무결성은 DB 제약으로도 지킵니다: `CHECK` (상태 값 목록, `checkName`), `UNIQUE(posting_id, user_id)` (같은 공고 중복 지원 차단, `uq_applications_posting_user`).

> 칸 이름은 테이블 정의서를 따라 snake_case.

`schemas.py` — **주문서 양식** 화면이 보내는 요청이 **올바른 모양인지** 검사해요. 예: `SignupRequest` 는 이름·이메일·비밀번호·약관 동의가 다 있어야 통과.

> 🔧 **전문 설명** — Pydantic v2 모델. 공통 부모 `Strict` 가 정의되지 않은 필드를 거부(`extra="forbid"`) 해 **대량 할당(mass assignment)** 을 막고, 🟫 `Field(pattern=…, max_length=…)` 로 입력 길이·형식을 경계에서 검증합니다. 예: `SignupRequest.phone` 은 `^01[0-9]-\d{3,4}-\d{4}$` .

📏 규칙판 (`domain/`)

`domain/status.py` — 보드게임 말판

- `transition` — "작성 중 → 준비됨"처럼 **허락된 칸으로만** 말을 옮겨요. 아니면 오류.
- `judgeDisconnect` — 끊겼을 때 '나가요' 쪽지를 보고 장애인지 본인이 나간 건지 정해요. 모르면 **지원자에게 유리하게 장애**.
- `nextAction` — 마이페이지 카드에 어떤 버튼(이어 쓰기·점검·이어가기…)을 보여 줄지 정해요.

> 🔧 **전문 설명** — 유한 상태 기계(FSM): `ALLOWED: dict[상태, set[다음 상태]]` 를 단일 진실 원천으로 두고 `transition()` 만 상태를 바꿉니다. 불법 전이는 `TransitionError` . `judgeDisconnect` 는 최근 이벤트 집합만 받는 순수 함수라 DB 없이 테스트됩니다.

`domain/scoring.py` — 계산기

- `itemScores` → `interviewScore` → `totalScore` — 항목 점수를 모아 면접 점수, 총점을 계산해요. 0.1+0.2 오차가 없는 🐍 `Decimal` 로 계산해요.
- `matchQuote` — AI가 가져온 근거가 답변에 진짜 있는지 찾아요 (띄어쓰기·문장부호 차이는 봐줘요).

> 🔧 **전문 설명** — 항목 점수 = 그 항목을 본 문항 점수 평균 ÷ 5 × 100, 면접 점수 = Σ(항목 점수 × 가중치), 총점 = 코딩 사용 시 비율 가중 합(EVL-03·04). 모두 🐍 `Decimal` + 🐍 `quantize(0.01, ROUND_HALF_UP)` 로 이진 부동소수점 오차(69.999…)를 없앱니다. `matchQuote` 는 정규화(공백·문장부호 제거) 후 정확 일치 → 없으면 `closestSpan` 으로 유사도 0.9 이상 구간을 찾아 **답변 원문 구절**을 돌려줍니다(E17).

`domain/question_text.py` — 질문 글 검사관 (2026-11-10)

- `textProblem` — AI 가 만든 질문 글을 보고 문제를 찾아요: 이상한 글자(이모지·영어 조각), "1문장으로"·"간단히" 같은 답하는 방법 지시, 한 번에 두 가지 묻기, 질문으로 안 끝남, "~는가요" 같은 어색한 말끝, 100자 넘음.
- `cleanQuestion` — 질문 끝에 묻어 나온 따옴표·괄호 부스러기를 털어 내요.
- `isYesNo` — "예/아니오"로 끝나 버리는 질문인지 알아봐요("무엇·어떻게·왜"가 없고 "설명해 주세요"도 아닌 질문).

> 🔧 **전문 설명** — DB·AI 없는 순수 함수 모음이라 단위 테스트(`test_question_text.py`)로 빠르게 검증됩니다. 문제 코드는 우선순위 순서로 하나만 돌려줍니다(foreign_script → junk_text → meta_phrase → compound_question → not_question → awkward_ending → too_long). 본질문(`question_compose.checkGenerated`)과 꼬리질문(`interview/graph.validateFollowup`)이 같은 함수를 써서 기준이 어긋나지 않습니다. 엑셀 질문은행·예비 질문에는 적용하지 않습니다(사용자 결정, C-16).

`domain/password_policy.py` — 비밀번호 규칙표

- `checkPassword` — 8~64자, 영문·숫자·특수문자 섞기 같은 규칙을 어기면 무엇을 어겼는지 알려 줘요.

> 🔧 **전문 설명** — 규칙: 길이 8~64, 영문·숫자·특수문자 각 1자 이상, 이메일 로컬 파트 포함 금지, 같은 문자 4회 연속 금지. 가입·변경·재설정이 같은 함수를 쓰고, 프론트 `PasswordChecklist` 도 같은 규칙을 보여 줍니다(서버가 최종 판단).

🛎 카운터 (`routers/`) — 주소별 창구

`routers/auth.py` — 회원 창구

- `signup` (회원가입), `login` (로그인 — 5번 틀리면 잠시 막힘), `logout` , `passwordReset` (비밀번호 찾기 메일), `withdrawMe` (탈퇴).

> 🔧 **전문 설명** — 로그인 실패는 계정이 아닌 **IP 단위**로 세어 `login_fail_limit` (기본 5) 초과 시 429 `LOGIN_BLOCKED` (계정 잠금 공격으로 남의 계정을 막지 못하게). 비밀번호 재설정은 계정 존재 여부와 상관없이 같은 응답(202)을 줘 이메일 열거를 막습니다. 토큰은 Redis 에 SHA-256 해시 키로 30분 TTL 저장, 사용 시 `DELETE` 결과로 1회만 통과(동시 클릭 경쟁 방지), 재설정 뒤 그 사용자의 모든 세션을 지웁니다. 로그인 쿠키는 `HttpOnly; SameSite=Lax` .

`routers/postings.py` — 공고 게시판 (지원자용)

- `listPostings` / `getPosting` — 공고 목록과 상세.

> 🔧 **전문 설명** — 인증 없이 열리는 읽기 전용 API. 게시(`published`) 공고만 노출하고 `postingView` 로 내부 칸(평가 기준 세부 등)을 걸러 냅니다.

`routers/applications.py` — 지원 창구

- `createApplication` — [지원하기]. 공고 하나에 한 번만.
- `consent` — 동의 5가지 받기.
- `uploadResume` → `acceptUpload` — 이력서 파일을 받아 크기·형식을 보고 주방(추출)에 넘겨요.
- `submitResume` — 이력서 확정 + 질문 준비 시작 (3장 장면 1).

> 🔗 **전문 설명** — `createApplication` 은 🟫 `with_for_update` 직렬화 + DB `UNIQUE` 2중 방어. 업로드는 🐍 `asyncio.Semaphore` 로 동시 2건, `detectType` 으로 매직 넘버(파일 앞 바이트) 검사, 상한(`upload_max_mb`) 초과는 413. 추출은 비동기로 돌고 화면은 `extract-status` 를 폴링합니다. 제출 뒤 수정은 409 `RESUME_LOCKED`.

`routers/interview.py` — 면접 창구

- `precheck` — 환경 점검 결과 받기.
- `startInterview` / `resumeInterview` — 면접 시작, 이어가기.
- `interviewSocket` — 면접 전화선(WebSocket) 연결 → `LiveSession` 에게 넘김.
- `codingRun` / `codingAutosave` / `codingSubmit` — 코딩테스트 실행·자동 저장·제출.

> 🔗 **전문 설명** — REST(점검·시작·이어가기·코딩) + WebSocket(진행) 혼합. `interviewSocket` 은 `Origin` 헤더가 `PUBLIC_ORIGIN` 과 같을 때만 받고(교차 사이트 WebSocket 하이재킹 방지), `registry.claimSession` 으로 세션 ID 당 연결 1개만 허용합니다. `beaconEvent` 는 🌐 `sendBeacon` 이 Content-Type 을 고를 수 없어 본문을 직접 파싱합니다. 코딩 API 는 남은 시간이 0이면 서버가 자동 제출(`autoSubmitIfOver`)합니다.

`routers/me.py` — 마이페이지 창구

- `myApplications` (지원 내역), `myResult` (결과, 공개 뒤에만), `notifications` (🔔 알림).

> 🔗 **전문 설명** — 결과는 `release_status == released` 이고 공개 시각이 지났으며 `purge_at` 전일 때만 반환(`isReleased`). 지원자용 결과에는 근거 인용·합불 사유 원문이 아닌 요약·강점·개선점만 담습니다(RPT-03).

`routers/admin.py`, `admin_ops.py`, `admin_questions.py` — 관리자 창구

- 공고: `createPosting`, `patchPosting`, `draftAi` (AI 초안), `draftAiRevise` (AI 에게 한 칸 다시 쓰기), `publish` (게시 — 게시하면 그 공고로 질문 후보 만들기도 시작), `close` (마감).
- 지원자: `applicants` (목록), `applicantDetail` (상세), `compare` (비교), `passed` (1차 합격자).
- 평가 지연: `reprocess` (다시 돌리기), `manualDecision` (직접 결정).
- 운영: `stats` (통계), `opsStatus` (운영 현황), `putSetting` (동시 면접 수), `releaseLoginBlock` (로그인 차단 풀기).
- 질문: `listQuestions` (출처와 적용 범위 `scopeView` 를 같이 보여 줌), `approve` ([추가]), `reject` ([제외]·[빼기]).

> 🔗 **전문 설명** — 모든 엔드포인트가 🟫 `Depends(requireAdmin)`. 변경 행위(게시·마감·직접 결정·설정 변경·차단 해제)는 `AuditLog` 에 남기되, 지원자 평가 내용(메모 본문·결정 사유)은 감사 로그에 넣지 않아 개인정보 노출면을 줄입니다. 지원자 이메일은 운영 현황에서 `maskEmail` 로 가립니다.

🍳 주방 (`services/`)

`question_compose.py` — 질문 요리사 (3장 장면 1)

- `compose` — 질문 10개 만들기 전체. `ensureComposed` — 면접 시작 때 아직이면 조금 기다리고, 넘으면 AI 없이 채워요.
- `checkGenerated` — 만든 질문 검사. `bankFallback` — 질문은행에서 고르기. `lastResort` — 최후의 고정 문장.
- `synthesizeAll` — 질문 음성(TTS)을 미리 만들어 둬요.
- `scopeFor` — 질문에 **어디서 쓸 수 있는지** 꼬리표를 붙여요: 회사 질문은 "이 공고만", 직무 질문은 "이 직무만", 인성 질문은 "어디서나".
- `savePending` — 면접 때 AI 가 만든 좋은 질문을 꼬리표와 함께 '검토 대기'로 모아 둬요(관리자가 [추가]해야 씀).

> 🔧 **전문 설명** — LangGraph 📦 `StateGraph(SlotState)` : `pick → examples → generate → check → (accept | regenerate | bank)` 조건부 간선. 중복 방지는 3중: 정규화 문자열 비교(`sameText`), 임베딩 코사인 0.9(`maxSimilarity`), 엑셀 `duplicate_group` . 사용 횟수는 `INSERT … ON CONFLICT` 로 경쟁 없이 +1. LLM 입력에는 `resumeText` 가 이름·연락처를 뺀 요약만 넣습니다.

`rag.py` — 비슷한 질문 찾는 사서

- `embed` — 문장의 '뜻'을 숫자 목록(벡터)으로 바꿔요.
- `similar` — 뜻이 가장 가까운 기존 질문을 찾아요. `cosine` — 두 뜻이 얼마나 가까운지 숫자 하나로 (1에 가까울수록 비슷).

> 🔧 **전문 설명** — 임베딩 서버(BGE-M3, 1024차원) HTTP 호출 → pgvector `vector` 칸에 저장, 검색은 코사인 거리. 응답은 개수·차원을 검사(`parseResponse`)하고 실패는 `EmbedUnavailable` 로 올려 상위가 질문은행으로 대체합니다. SQLite(테스트)에서는 `supported() == False` 로 벡터 기능을 끕니다.

`resume_extract.py` — 이력서 번역가

- `extractUpload` — 읽기 → (스캔본이면) OCR → AI 정리 → 저장, 전체 흐름.
- `detectType` — 이름만 `.pdf` 로 바꾼 가짜 파일을 걸러요.
- `structure` — AI가 글을 학력·경력·기술 칸에 나눠 담아요.
- `lowConfidenceFields` — 흐릿하게 읽힌 칸을 표시해서 지원자가 확인하게 해요.

> 🔧 **전문 설명** — 파이프라인: 형식 검사 → 격리 프로세스(`pdf_box`)로 텍스트층 추출(PyMuPDF → pdfplumber) → `isReadable` 이 실패한 쪽만 렌더링 후 OCR(PaddleOCR) → `repairDateRanges` · `splitCoverLetter` 전처리 → LLM 구조화(`StructuredResume` 스키마) → `fillBlanks` 병합. OCR 신뢰도 낮은 줄에서 온 칸 경로(`career[0].company`)를 돌려 화면이 강조합니다. 동시 추출은 세마포어로 제한.

`pdf_box.py` — **유리방 PDF 읽개** PDF는 **따로 떨어진 작은 프로세스**에서 읽어요. 이상한 PDF가 터져도 서버는 멀쩡해요.

- `readPages` — 쪽마다 글 뽑기. `renderPage` — OCR용 그림 만들기.

> 🔧 **전문 설명** — 신뢰할 수 없는 파일 파싱을 **프로세스 격리**합니다: `subprocess` 로 stdin 에 PDF 를 넣고, 리눅스에서는 `resource.setrlimit` 으로 메모리·CPU 시간을 제한하며 부모가 타임아웃을 겁니다. 파서 크래시·폭탄 PDF 가 API 프로세스에 영향을 주지 않습니다.

`evaluation.py` — **채점 선생님** (3장 장면 4)

- `evaluate` — 채점 전체. `scoreQuestion` — 질문 하나 채점. `computeScores` — 점수 계산.
- `redact` — 이름·연락처 지우기. `fence` / `instructionHits` — 답변 속 "만점 줘" 같은 **몰래 명령** (프롬프트 인젝션) 막기.
- `writeReason` / `writeFeedback` — 결과 이유, 강점·개선점 글.
- `decideManually` — 관리자 직접 결정을 저장해요.

> 🔧 **전문 설명** — 입력 준비(`loadInput` : 지원 시 잠근 기준 버전만) → `redact` (PII 제거) → `fence` (꺾쇠 치환으로 구분 태그 탈출 방지) → `instructionHits` (채점 조작 문구 탐지 → 관리자 확인) → T5 문항 채점(`T5Out` 스키마, 근거 `max_length` 제한) → `checkT5` → `computeScores` → T10 이유(`checkT10` 검증) → T6/T7/T8 요약·피드백·코드 리뷰 → `save` (행 잠금 단일 트랜잭션). 실패 유형을 `AiUnavailable` (재시도)· `EvalFailed` · `NeedsReview` (사람 검토)로 나눠 처리합니다.

`llm.py` — **AI 전화기**

- `structured` — AI에게 묻고, 대답이 **정해진 모양(JSON)** 인지 검사해요. 시간 초과·모양 틀림은 오류.
- `laneUrl` — 빠른 4B 서버로 갈지, 꼼꼼한 8B 서버로 갈지.

> 🔧 **전문 설명** — llama.cpp 서버(OpenAI 호환 API, Q4_K_M 양자화 GGUF 모델)를 LangChain `ChatOpenAI` 로 호출. `structured` 는 `with_structured_output(schema, method='json_schema')` 로 JSON 스키마에 맞춘 출력을 받고 Pydantic 으로 다시 검증해 **형식을 두 번 확인**합니다. lane `rt` =Qwen3 4B(GPU, 실시간), `eval` =Qwen3 8B(CPU, 결과 이유).

`speech.py` — **귀와 입**

- `transcribe` — 소리 → 글 (STT). `synthesize` — 글 → 소리 (TTS).

> 🔧 **전문 설명** — 내부 STT(Whisper 계열, GPU)·TTS(CPU) 서비스 HTTP 클라이언트. STT 입력은 16kHz mono PCM16, 출력은 글·단어 타임스탬프·처리 시간. 실패는 `SpeechError` 로 올려 엔진이 재시도/장애 처리를 정합니다.

`admission.py` — 놀이기구 번호표

- `tryAcquire` — 자리 하나 잡기. Redis 안에서 **끼어들 틈 없이 한 번에**(Lua) 세요.
- `release` — 자리 돌려주기. `reconcile` — 1분마다 숫자를 장부와 맞춰요.

> 🔗 **전문 설명** — 동시 면접 상한 N 을 Redis 카운터 `interview:active` 로 관리. `ACQUIRE_LUA` 가 `GET → 비교 → INCR` 을 서버 측에서 원자적으로 실행해 **TOCTOU 경쟁**을 없앱니다. 커밋 전 입장은 `interview:pending:{id}` (TTL 60초)로 표시해 1분 주기 `reconcile` 이 놓치지 않습니다(E10).

`coding.py` · `coding_bank.py` · `sandbox_runner.py` — 코딩테스트 담당

- `pickProblems` — 지원자마다 기초 1 · 응용 1 문제를 골라요.
- `runExamples` — [실행] 버튼: 예시 입력으로 돌려 보기.
- `finishCoding` — 제출(또는 시간 끝) → 숨겨진 테스트 채점 예약.
- `run` — 지원자 코드를 **유리 상자**(인터넷 끊김, 메모리 256MB, 10초) 안에서 돌려요.

> 🔗 **전문 설명** — 문제 배정은 지원 ID 기반 결정적 선택(같은 지원 = 같은 문제). 코드 실행은 api 가 직접 하지 않고 Celery `sandboxRunTask` 로 worker-sandbox 에 위임(api 컨테이너에 Docker 소켓 없음). 숨겨진 테스트 채점은 5분 중복 억제 키로 멱등 처리합니다.

`postings.py` · `posting_draft.py` — 공고 담당

- `validateForPublish` — 게시 전에 빈칸이 없나 확인. `updatePosting` — 고치기(게시 뒤엔 잠긴 칸 있음). `closePosting` — 마감.
- `start` / `generate` — AI 공고 초안. `biasWarnings` — 나이·성별·종교 같은 차별 표현 찾기.
- `buildDraftGraph` — AI 공고 초안을 **순서도**로 만들어요: ① 같은 직무의 예전 공고를 참고하고 → ② 초안을 쓰고 → ③ 칸마다 검사(차별 표현? 예전 공고 베끼기?) → 문제 있는 칸만 다시 써요.
- `startRevision` / `revise` — 관리자가 "이 칸만 다시 써 줘" 하면 그 칸만 고쳐 화면에 돌려줘요. 저장은 관리자가 [임시저장]을 눌러야 돼요.

> 🔗 **전문 설명** — 평가 기준은 버전 행(`EvaluationCriteria`)으로 쌓고 첫 응시 시작 시 잠급니다 (`locked`) — 이미 응시한 사람과 다른 기준으로 채점되지 않게. 게시 뒤 직무·기간 같은 칸은 수정 불가(409), 본문 수정은 `PostingRevision` 이력과 지원자용 요약(`revisionSummary`)을 남깁니다. AI 초안은 LangGraph `StateGraph(DraftState)` (retrieve → draft → check → rewrite ↺ → save / deliver / fail)로, 체크포인트 thread `posting-draft:{공고 id}` 에 처음 입력·참고 공고를 남겨 두었다가 한 칸 다시 쓰기 때 같은 thread 로 다시 실행합니다(C-15). 상태에는 지원자 개인정보가 없습니다.

`posting_questions.py` — 공고를 보고 질문 미리 만들어 두기 (2026-11-10)

- `schedule` → `generateForPosting` — 공고를 **게시하면** 뒤에서 질문 후보를 만들어요. 공고의 업무·자격 줄마다 직무 질문 1개(최대 6개), 회사 질문 3개(지원 동기 · 회사가 하는 일 · 조직 문화로 초점을 나눠서).
- `makeOne` — 질문 1개를 만들고 면접 질문과 같은 검사를 해요. 이미 비슷한 질문이 있으면 버려요.

- `loadWebCandidates` — 인터넷 면접 질문 자료를 참고해 새로 쓴 후보 파일을 불러와요(명령 `load_web_questions`).
- 모두 '검토 대기'로만 들어가요. 관리자가 질문 관리 화면에서 [추가]해야 면접에 쓰여요.

> 🔗 **전문 설명** — 게시 API(`publish`)가 커밋한 뒤 🐍 `asyncio` 작업으로 띄우고(`tasks` 집합에 참조 보관), 예외는 잡아서 로그만 남깁니다 — 부가 기능이 게시를 막지 않게. 프롬프트·검사는 `question_compose.ragPrompt · checkGenerated` 를 재사용하고, 이력서 칸은 '(없음)'이라 지원자 개인정보가 들어가지 않습니다. 중복은 정규화 글(`duplicate_group`) 일치와 임베딩 코사인 `BANK_DUPLICATE_SIMILARITY` (0.8)로 막습니다. 감사 기록 `question.posting_candidates` · `question.web_candidates`.

`login_block.py` — 경비원

- `recordFailure` — 로그인 실패 세기. 같은 곳에서 5번이면 차단. `release` — 관리자가 풀어 주기.

> 🔗 **전문 설명** — 키 = 클라이언트 IP 의 해시(`guardKey`). `X-Forwarded-For` 는 신뢰 프록시에서 온 요청일 때만 믿어 IP 위조를 막고(`clientIp`), 예외 대역(`loginExemptIps`)을 지원합니다. 집계 창 안 실패가 상한을 넘으면 `blocked_until` 까지 429, 관리자는 문의 코드로 해제(감사 기록).

`notify.py` — 우체부

- `create` — 🔔 알림 만들기. `sendNow` — 메일 1통 보내기. `retryOrFail` — 실패하면 다시, 그래도 안 되면 실패 기록.

> 🔗 **전문 설명** — 알림 행은 업무 트랜잭션 안에서 만들고, 메일은 **커밋 뒤** 큐에 넣는(transactional outbox 와 비슷한) 순서라 롤백된 알림이 메일로 나가지 않습니다. SMTP 실패는 `RETRY_DELAYS_SEC` 간격 재시도 후 `failed` 로 남겨 대시보드에서 재처리합니다.

`storage.py` · `recording.py` — 파일 창고와 녹화 담당

- `putObject` / `getObject` — 파일을 잠가서 넣고, 풀어서 꺼내요.
- `saveChunk` — 10초짜리 녹화 조각을 차례로 저장해요.

> 🔗 **전문 설명** — S3 호환(RustFS) 📦 `boto3` 클라이언트. 서버 측 암호화에 기대지 않고 **애플리케이션 계층에서 AES-GCM 암호화**하며, 객체 키를 AAD(추가 인증 데이터)로 묶어 다른 경로로 바꿔치기한 객체는 복호화 단계에서 거부됩니다.

`retention.py` · `withdrawal.py` — 청소부

- `purgeDue` — 보관 기간(결과 공개 +7일) 지난 데이터 지우기. `purgeRecordings` — 녹화는 3일.
- `archiveStats` — 지우기 전에 **통계 숫자만** 남겨 둬요.
- `withdrawApplication` / `withdrawAccount` — 동의 철회·탈퇴하면 바로 지워요.

> 🥓 **전문 설명** — 보관 기한(`purge_at` = 결과 공개 + 7일, 녹화 = 응시 끝 + 3일) 지난 데이터를 매일 03:00 삭제. 파일 먼저, DB 나중 순서로 고아 파일을 막고, 건별 실패는 다음 날 재시도합니다. 삭제 전 `archiveStats` 로 익명 집계만 보존하고 동의 기록은 증빙으로 남깁니다.

`question_bank.py` — 질문은행 사서

- `loadAll` — 엑셀 질문들을 장부에 넣어요. `dedupeExcel` — 뜻이 겹치는 질문을 정리해요.

> 🥓 **전문 설명** — 🟫 `openpyxl` read-only 모드로 엑셀을 스트리밍 읽기. (시트, 행 번호) 키로 중복 적재를 막고, 다른 회사 이름·무관한 질문은 정규식으로 제외합니다. `dedupeExcel` 은 임베딩 생성 뒤 의미 유사 중복을 `excluded` 로 바꿉니다.

🎙️ 면접실 (`interview/`)

`interview/engine.py` — 면접 진행 직원 `LiveSession` (3장 장면 2·3)

- `serve` — 일 시작. `send` — 화면에 말 전하기.
- `onText` — 화면에서 온 쪽지(심장 박동, 화면 이탈, 답변 완료…)를 종류별로 처리.
- `onAway` — 화면 이탈: 1번째 경고, 2번째 취소. 3초 안에 온 신호는 한 번으로 셈.
- `watchdog` — 전화선 지키기. `judgeDeviceEnded` — 장치가 꺼지면 3초 기다렸다 판정.
- `pauseForFault` — 장애 중단. `endWith` — 응시 끝내기.

> 🥓 **전문 설명** — 연결당 1개의 `LiveSession` 이 상태(`idle → preparing → answering → processing → coding → ended`)를 가지며, `send` 는 🐍 `asyncio.Lock` 으로 동시 송신을 직렬화합니다. 화면 이탈은 `AWAY_MERGE_SEC = 3` 안의 신호를 1회로 병합(탭 숨김 + 포커스 잃음 이중 신호), 장치 종료는 `DEVICE_END_GRACE_SEC = 3` 뒤 판정해 unload 비콘이 먼저 도착할 시간을 줍니다(E18).

`interview/graph.py` — 면접 순서도

- `buildInterviewGraph` — 준비 → 질문 → 답변 → 꼬리질문 → 다음… 순서를 그림처럼 정해 둬요 (LangGraph).
- `buildFollowupGraph` — 꼬리질문 순서도. `validateFollowup` — 꼬리질문 검사.

> 🥓 **전문 설명** — 면접 진행을 LangGraph 그래프로 선언하고, 실제 부작용(DB·WebSocket)은 `Runner` 프로토콜 구현체(`LiveSession`)에 위임합니다 — 그래프 로직을 가짜 러너로 단위 테스트할 수 있습니다. 답변 대기는 🟫 `interrupt()` 로 멈추고 체크포인트에서 재개, `RECURSION_LIMIT` 으로 무한 루프를 막습니다.

`interview/lifecycle.py` — 면접 기록원

- `logEvent` — 끊김·이탈 같은 일을 시각과 함께 적어요.
- `resumesLeft` — 남은 이어가기 횟수 (3번 – 지원자 쪽 끊김 수).

- `markInterrupted` · `finish` · `enterCoding` — 장애 중단, 끝, 코딩으로 넘어가기.

> 🔧 **전문 설명** — 응시 수명 주기의 부작용 모음: 이벤트 기록(`session_events`), 장애 중단 (`markInterrupted` — 지원자 원인 끊김이 `resume_limit` 에 닿으면 종료, 서버 원인 `SERVER_SIDE_REASONS` 는 세지 않음), 종료(`finish` — 자리 반환 + 채점 큐잉), 코딩 타이머 일시정지·재개.

`interview/registry.py` — 지금 일하는 직원 명단

- `register` / `get` — 지원서마다 지금 면접 중인 직원을 찾아요. `terminateMany` — 공고 마감 때 한꺼번에 끝내요.

> 🔧 **전문 설명** — 프로세스 메모리의 `dict[지원 ID, LiveSession]` . 같은 지원 건에 새 연결이 오면 이전 세션을 `replaced` 로 끝냅니다. api 프로세스 하나를 전제로 한 설계라, api 를 여러 개로 늘리려면 프로세스 사이에 공유되는 명부가 따로 필요합니다(현재 범위 밖).

`interview/checkpoint.py` — 저장 포인트 게임 세이브처럼, 면접 순서도가 **어디까지 했는지** 저장해 둬요.

> 🔧 **전문 설명** — 운영은 📦 `AsyncPostgresSaver` (psycopg 연결 풀), 테스트는 📦 `InMemorySaver` . `thread_id` = 지원 ID 라 재연결·서버 재시작 뒤에도 같은 그래프 상태에서 재개합니다. 응시가 끝나면 `deleteThread` 로 지웁니다.

🌙 뒷주방 (`worker.py` , `jobs.py`)

`worker.py` — 밤에도 일하는 작업자 (Celery) | 작업 | 언제 | 하는 일 | |---|---|---| | `evaluateTask` | 응시가 끝나면 | 채점 | | `sweepTask` | 5분마다 | 빠진 채점 다시 넣기, 결과 공개(`releaseDue`), 평가 지연 정리 | | `sendMailTask` | 알림이 생기면 | 메일 보내기 | | `remindersTask` | 매시 정각 | 마감 임박 안내 | | `purgeTask` | 매일 03:00 | 오래된 데이터 지우기 | | `purgeNoticeTask` | 매일 09:00 | 삭제 2일 전 관리자 안내 | | `purgeRecordingsTask` | 매시 30분 | 기한 지난 녹화 지우기 | | `sandboxRunTask` · `gradeCodingTask` | 코딩 실행·채점 때 | 유리 상자에서 코드 돌리기 |

> 🔧 **전문 설명** — Celery 앱(브로커·결과 백엔드 = Redis). 큐를 나눠 `evaluate` (채점, `-c 1`), `default` (메일·점검, `-c 2`), `sandbox` (코드 실행, `-c 2` , 이 작업자만 Docker 소켓 보유)로 격리합니다. 주기 작업은 `beat_schedule` (`sweep` 300초, `purge` 03:00 등). 채점은 Redis 잠금 키로 같은 건 중복 실행을 막고, AI 서버 다운은 실패로 세지 않고 `reschedule` 로 미룹니다.

`jobs.py` — 1분 알람

- `runForever` — 서버 안에서 1분마다: `sweepOrphans` (주인 없는 면접 정리), `releaseHeldSeats` (끊긴 지 10분 지난 자리 돌려주기), `closeExpiredPostings` (마감 시각 지난 공고 닫기), `reconcile` (자리 숫자 맞추기), `regradePending` (못 끝낸 코딩 채점 다시).

> 🔧 **전문 설명** — Celery 와 별개로 api 프로세스 안 🐍 `asyncio` 루프에서 60초마다 도는 가벼운 정리 작업. 각 주기를 try/except 로 감싸 한 번의 실패가 루프를 멈추지 않게 합니다. 진행 중 WebSocket 세션을 끊어야 하는 일(공고 자동 마감)이 있어 같은 프로세스에 둡니다.

🔧 관리 도구

`manage.py` — 설치할 때 쓰는 명령들. `init` (첫 설치·관리자 계정), `loadQuestions` (질문 엑셀 넣기), `embedQuestions` (질문 뜻을 숫자로).

> 🔧 **전문 설명** — `python -m app.manage <명령>` CLI. `init` 은 멱등(이미 있으면 건너뜀)이라 여러 번 돌려도 안전합니다. 관리자 초기 비밀번호는 `must_change_password` 가 켜진 채 만들어져 첫 로그인 때 변경을 강제합니다(`activeUser`).

`migrate.py` — 장부에 새 칸·규칙(UNIQUE 등)을 덧붙여요. **`devtools.py`** — 개발자가 잠긴 글을 서버 안에서만 풀어 보는 도구예요.

---

5. 프론트엔드 파일 소개 — 화면들

5-1. 화면 파일 (`frontend/app/`) — 주소 = 파일

주소	파일	무슨 화면?	중요한 조각
`/jobs`	`app/jobs/page.tsx`	공고 목록	`JobList` (검색·거르기)
`/jobs/[postingId]`	`app/jobs/[postingId]/page.tsx`	공고 상세	`JobDetail` ([지원하기])
`/login` , `/signup`	`app/login` , `app/signup`	로그인, 회원가입	`LoginForm` , `SignupForm`
`/apply/…/consent`	`app/apply/[applicationId]/consent`	① 동의	`Consent`
`/apply/…/resume`	`…/resume` , `…/resume/confirm`	② 이력서 고르기·확인	`ResumeMethod` , `Confirm`
`/apply/…/precheck`	`…/precheck`	③ 환경 점검	`PrecheckStep` → `Precheck`
`/apply/…/rules`	`…/rules`	④ 시작 안내	`Rules` ([면접 시작])
`/exam/[applicationId]`	`app/exam/[applicationId]/page.tsx`	**면접 화면**	`Exam` (가장 큰 화면, 600줄)
`/exam/…/resume`	`app/exam/[applicationId]/resume`	이어가기	`Resume`
`/mypage`	`app/mypage/page.tsx`	마이페이지	`Applications` , `Results` , `Account`
`/mypage/results/…`	`app/mypage/results/[applicationId]`	결과 상세	`ResultDetail`
`/admin/postings`	`app/admin/postings/…`	공고 관리·만들기	`PostingEditor` , AI 초안 `Chips` , 한 칸 다시 쓰기 `AiRewrite`

주소	파일	무슨 화면?	중요한 조각
`/admin/applicants`	`app/admin/applicants/…`	지원자 목록·상세	`Applicants`, `ApplicantDetail`, `Report`
`/admin/compare`	`app/admin/compare`	지원자 비교	`Radar` (레이더 그림)
`/admin/dashboard`	`app/admin/dashboard`	대시보드	`Bars` (막대그래프)
`/admin/questions`	`app/admin/questions/page.tsx`	질문 관리 — AI 가 만든 질문 검토 (출처·적용 범위 표시)	`Questions`
`/admin/settings`, `/admin/next-stage`		운영 설정, 다음 전형	

> 🔧 **전문 설명** — Next.js App Router 의 파일 시스템 라우팅: `app/**/page.tsx` 가 라우트, `[param]` 은 동적 세그먼트(📦 `useParams`). 페이지 기본 내보내기(`…Page`)는 📦 `<Suspense>` 경계로 📦 `useSearchParams` 를 쓰는 본문 컴포넌트를 감쌉니다. API 는 같은 출처 `/api/*` 를 `next.config` rewrites 로 FastAPI 에 넘겨 쿠키(HttpOnly, SameSite)와 CORS 문제를 피합니다.

`page.tsx` 안의 `…Page` 함수는 "이 주소로 오면 여기서 시작"하는 **입구**예요. 실제 내용은 같은 파일의 다른 조각(`Exam`, `Consent` …)에 있어요.

5-2. 같이 쓰는 조각 (`frontend/components/`)

파일	별명	중요한 것
`Providers.tsx`	가게 전체 방송실	`useApp` — 지금 로그인한 사람, 토스트 알림. 로그인이 끝나면(401) 알려 줘요
`Shell.tsx`	건물 뼈대	`Gnb` (위 메뉴), `useGuard` (역할 문지기), `MobileBanner` (모바일 안내)
`AdminShell.tsx`	관리자 사무실 뼈대	왼쪽 메뉴, 관리자만
`ApplyFlow.tsx`	지원 단계 안내원	`Stepper` (①②③④ 표시), 상태가 안 맞으면 맞는 단계로 돌려보내기
`Precheck.tsx`	환경 점검 기사	마이크·카메라·스피커·네트워크·AI 서버 점검. 요청을 **줄 세워 차례로** 보내요
`exam/Overlays.tsx`	면접 중 덮개 창	경고, 장애 중단(남은 이어가기 횟수), 종료 안내
`exam/Coding.tsx`	코딩테스트 화면	30초마다 자동 저장, [실행], [제출]
`PostingEditor.tsx`	공고 작성 3단계	`weightError` (가중치 합 100%), `criteriaError` (합격 기준 빈칸), `AiRewrite` (AI 에게 한 칸 다시 쓰기)
`ResumeForm.tsx`	이력서 입력 폼	흐릿하게 읽힌 칸 강조
`Ui.tsx`	레고 상자	`Button`, `Modal`, `Field`, `Spinner`, `ErrorBox`
`NotificationBell.tsx`	🔔 종	알림 목록·읽음·숨기기

> 🔧 **전문 설명** — 공통 컴포넌트는 Context(`Providers` 의 `useApp`, `ApplyFlow` 의 `useApplication`)로 데이터를 내려 prop drilling 을 줄입니다. 접근성: `Modal` 은 포커스 트랩· `aria-labelledby`, 응시 중 덮개는 ESC 로 닫히지 않음, 그래프(`Bars`, `Radar`)는 같은 값을 표로도 제공합니다. `Precheck` 는 서버 점검 요청을 Promise 체인 큐(`checkQueue`)로 직렬화해 늦게 온 응답이 통과 표시를 덮어쓰는 경쟁을 없앴습니다.

5-3. 도우미 (`frontend/lib/`)

파일	하는 일
`api.ts`	**모든 서버 요청의 문**. `api("/auth/login", {body})` 처럼 써요. 오류는 `{code, message}` 로 알려 줘요
`media.ts`	카메라·마이크·스피커. `openStream` (켜기), `routeToSpeaker` (고른 스피커로 소리 내기), `measureMbps` (인터넷 속도)
`pcm.ts`	답변 녹음기 `PcmRecorder` — 말하는 동안만 소리 조각을 서버로 보내요
`recording.ts`	카메라 녹화 — 10초 조각으로 잘라 올려요
`exam.ts`	면접 정보 보관, 창을 닫을 때 **'나가요' 쪽지**(`sendUnloadBeacon`) 보내기
`format.ts`	시각을 한국 시간으로 예쁘게 (`longDateTime` , `mmss` = 01:30)
`status.ts`	상태 이름표와 색 (`interviewing` → "면접 진행 중", `badge-blue`)

🔧 **전문 설명** — `api.ts` 는 🌐 `fetch` 래퍼(`credentials: "same-origin"` , `cache: "no-store"`)로 오류를 `{code, message}` 예외로 통일하고, `/auth/` 밖의 401 은 `UNAUTHORIZED_EVENT` 로 방송해 전역 로그아웃 처리를 합니다. 미디어는 🌐 `getUserMedia` (deviceId `ideal` 제약 — 장치를 빼도 다른 장치로 열림), 🌐 `AudioWorkletNode` (`/pcm-worklet.js`) 기반 PCM 캡처, 🌐 `MediaRecorder` (webm, 10초 timeslice), 스피커 지정은 🌐 `HTMLMediaElement.setSinkId` 를 씁니다.

---

6. 테스트 — 함수의 '사용 설명서'

함수가 맞게 움직이는지 궁금하면 **테스트**를 보세요. 테스트는 "이 함수는 이렇게 동작해야 해"라고 적어 둔 시험지예요.

- 백엔드: `backend/tests/test_….py` — 예: `test_scoring.py` 는 점수 계산, `test_device_end.py` 는 창 닫기 판정(E18), `test_question_text.py` 는 질문 글 검사, `test_posting_questions.py` 는 질문 적용 범위와 게시 때 질문 후보를 시험해요.
- 화면 전체: `e2e/tests/….spec.ts` — 진짜 브라우저로 지원부터 결과까지 눌러 봐요.
- 찾는 법: `Ctrl+Shift+F` 로 함수 이름을 `tests` 폴더에서 찾으면, 그 함수를 어떻게 쓰는지 예시가 나와요.

전체 테스트 파일 목록은 프로그램 명세서 5장에 있어요.

🔧 **전문 설명** — 백엔드 테스트는 pytest + SQLite 메모리 DB + fakeredis 로 외부 의존 없이 돌고, LLM·STT 는 가짜 객체로 바꿉니다(TDD: 실패하는 테스트 먼저). E2E 는 Playwright(Chromium)가 실제 스택(docker compose)에 가짜 카메라·마이크(`--use-fake-device-for-media-stream`)로 접속해 사용자 흐름 전체를 검증합니다.

부록 — 함수 전체 목록은 '프로그램 명세서'로 옮겼어요

모든 파일과 함수를 빠짐없이 적은 사전은 이제 따로 된 문서 **`프로그램명세서`**(같은 폴더, md · pdf)에 있어요. 실무에서 '프로그램 명세서'라고 부르는 문서 모양으로 정리했어요.

예전 이 가이드의 부록	프로그램 명세서
부록 표 읽는 법	1장 표 읽는 법
A-0 · B-0 폴더 지도	2장 프로그램 목록
부록 A 백엔드 함수 표	3장 백엔드 프로그램 명세
부록 B 프론트엔드 함수 표	4장 프론트엔드 프로그램 명세
부록 C 테스트 지도	5장 테스트 대응표

> 🔗 **전문 설명** — 이름·줄 번호·인자·부르는 곳은 코드에서 자동으로 뽑아(Python `ast`, TypeScript 구문 분석) 명세서에 다시 채웁니다. '하는 일'은 사람이 쓴 글이라 다시 뽑을 때도 그대로 둡니다. 실무 용어로는 API 레퍼런스 자동 생성 + 상호 참조(xref) + 요구사항 추적이에요.